Deep Learning with PyTorch - Technical Report

Neela Vyshali | Skill Set Go EduTech - AI/ML Internship - Week 2

1. Problem and data

This report compares three PyTorch models built for Week 2: a feed-forward network for tabular binary classification, a convolutional neural network (CNN) for image classification, and a set of controlled hyperparameter experiments on the CNN.

Task 2.1 used the Breast Cancer Wisconsin dataset (569 samples, 30 numeric features, malignant/benign). Tasks 2.2 and 2.3 used CIFAR-10 (60,000 32x32 RGB images across 10 classes; 45,000 train / 5,000 validation / 10,000 test for 2.2, a 10,000/2,000 subset for the faster 2.3 experiments).

2. Architecture and training setup

Task 2.1 - 30-16-8-1 feed-forward network with ReLU activations; BCEWithLogitsLoss; Adam (lr=0.001); batch size 32; 50 epochs. Final train loss 0.0308, final validation loss 0.0952, validation accuracy 95.61%.

Task 2.2 - SmallCNN: three convolution blocks (32-64-128 filters) with ReLU and max-pooling, followed by a fully-connected classifier with dropout 0.3 (356,810 parameters). CrossEntropyLoss; Adam (lr=0.001); batch size 64; 15 epochs. Best validation accuracy 79.12%, final test accuracy 78.78%.

Task 2.3 - the same SmallCNN architecture and a fixed 10,000/2,000 data subset were reused for every run, changing exactly one hyperparameter at a time from a baseline (Adam, lr=0.001, batch 64, 5 epochs): learning rate, optimizer type, batch size, and number of epochs.

3. CNN results (Task 2.2)

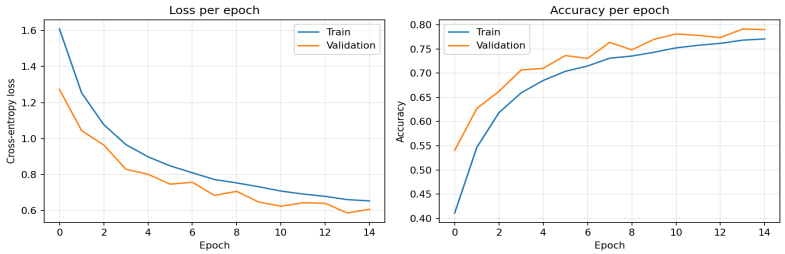
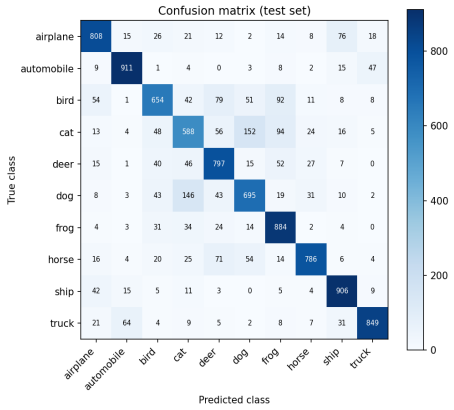


Figure 1. Training/validation loss and accuracy per epoch (15 epochs).



Class	Test acc.
automobile	91.1%
ship	90.6%
frog	88.4%
truck	84.9%
airplane	80.8%
deer	79.7%
horse	78.6%
dog	69.5%
bird	65.4%
cat	58.8%

Figure 2 / Table 1. Confusion matrix and per-class test accuracy. Cat (58.8%) and bird (65.4%) are confused most often with dog and other animal classes; automobile (91.1%) and ship (90.6%) are the most reliable classes.

4. Hyperparameter experiments (Task 2.3)

Ten runs (R01-R10) were planned by changing one setting at a time from the baseline: R02-R04 varied the learning rate (0.1, 0.01, 0.0001), R05-R06 compared plain SGD and SGD with momentum, R07-R09 varied batch size (32, 128, 256), and R10 extended training to 20 epochs to check for overfitting.

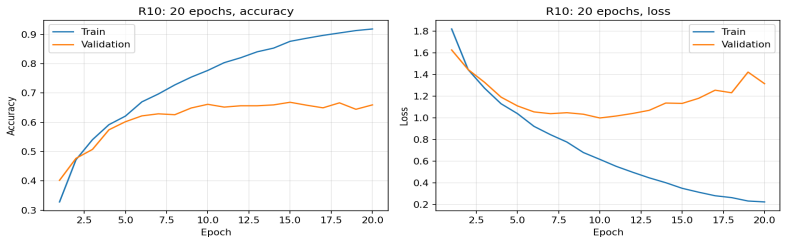


Figure 3. R10 (20 epochs) - the clearest signal in the experiment set. Training accuracy keeps climbing to about 92%, while validation accuracy plateaus around 65-66% after epoch ~10. Validation loss reaches its lowest point near epoch 10 (~1.0) and then rises again to ~1.4 by epoch 19, even as training loss keeps falling - a textbook overfitting pattern once training runs past the point the validation set supports.

5. Underfitting / overfitting

Task 2.1 shows a small, stable gap between train and validation loss (0.031 vs 0.095), consistent with reasonable generalisation on a small, clean tabular dataset. Task 2.2's 15-epoch run stays close to that pattern (train 76.8% vs. best validation 79.1%). Task 2.3's R10 run demonstrates what happens when training continues well past that point on a smaller data subset: a widening train/validation gap and rising validation loss are clear overfitting indicators.

6. Limitations and next steps

The full one-change-at-a-time sweep (R01-R09) was run but its numeric comparison table is not included in this report, since the output had not been exported at the time of writing. Task 2.3 also used a reduced 10,000-image subset rather than the full CIFAR-10 training set, to keep ten runs practical to execute.

Next steps:

- Add the full R01-R09 comparison table once the output is available, to confirm which learning rate, optimizer and batch size combination performs best rather than relying on R01 and R10 alone.
- Apply early stopping or a validation-based checkpoint to Task 2.2/2.3 to avoid the overfitting seen in R10 and recover a model closer to its best epoch.